

OPERATING SYSTEMS (UE24CS242B)

Unit 1 – System Calls for Process Management & Life Cycle of an I/O Request

System Calls for Process Management

System calls provide the mechanism through which a process requests services from the operating system related to process creation, execution, synchronization, and termination.

Process Identifiers (PID)

- Every process is assigned a unique process identifier (PID).
- PID is a non-negative integer.
- PID is the only universally known identifier that uniquely identifies a process.
- PIDs are often embedded into other identifiers to ensure uniqueness.

PID Reuse

- Although PIDs are unique at any instant, they are reused after processes terminate.
- UNIX systems use algorithms to delay PID reuse so that newly created processes do not get recently used PIDs, avoiding confusion.

fork() System Call

- An existing process creates a new process by calling fork().
- The newly created process is called the child process.

Return Values

- Child process → returns 0
- Parent process → returns PID of the child
- Error → returns -1

Behavior

- Child is an **exact copy** of the parent.
- Child receives a copy of:
 - Data segment
 - Heap
 - Stack
- Both parent and child continue execution from the instruction following fork().

```
#include <unistd.h>

pid_t fork(void);
```

vfork() System Call

- vfork() creates a new process **without copying the address space** of the parent.
- Assumes the child will immediately call:

- `exec()` **or**
- `exit()`

Key Characteristics

- Child shares the parent's address space temporarily.
- Guarantees that child runs first.
- Parent is suspended until the child:
 - Calls `exec()` or
 - Calls `exit()`

Return Values

- Child → 0
- Parent → PID of child
- Error → -1

`vfork()` is faster than `fork()` but must be used carefully.

exit() – Process Termination

A process can terminate normally in the following ways:

1. Returning from `main()`
2. Calling `exit()`
3. Calling `_exit()` or `_Exit()`
4. Returning from the start routine of the last thread
5. Calling `pthread_exit()` from the last thread

Abnormal Termination

A process can terminate abnormally by:

1. Calling `abort()`
2. Receiving certain signals
3. Last thread responding to a cancellation request

Kernel Actions on Termination

Regardless of termination type:

- Kernel closes all open file descriptors
- Kernel releases all memory used by the process

wait() and waitpid()

These system calls allow a parent process to synchronize with its child processes.

Possible Behaviors

A process calling wait() or waitpid() can:

1. **Block**
 - If all child processes are still running
2. **Return immediately**
 - If a child has already terminated and is waiting
3. **Return error**
 - If the process has no children

If called after receiving SIGCHLD, wait() returns immediately.

Return Values

- Returns PID of child on success
- Returns 0 if state hasn't changed
- Returns -1 on failure

waitid()

- waitid() allows finer control over which child process to wait for.
- Uses separate arguments instead of combining PID and process group.

Return Values

- 0 → success
- -1 → error

exec() Family of System Calls

- When a process calls an exec() function:
 - The current process image is completely replaced
 - New program starts execution at its main() function

Important Points

- **PID does NOT change**
- No new process is created
- Text, data, heap, and stack are replaced

Return Value

- Returns -1 on error
- Does not return on success

```
#include <unistd.h>

int execl(const char *pathname, const char *arg0, ... /* (char *)0 */ );
int execv(const char *pathname, char *const argv[]);
int execlp(const char *pathname, const char *arg0, ...
/* (char *)0, char *const envp[] */ );
int execve(const char *pathname, char *const argv[], char *const envp[]);
int execlp(const char *filename, const char *arg0, ... /* (char *)0 */ );
int execvp(const char *filename, char *const argv[]);
int fexecve(int fd, char *const argv[], char *const envp[]);
```

getpid() and getppid()

- getpid()
 - Returns the PID of the calling process
- getppid()
 - Returns the PID of the parent process

If the original parent has terminated:

- Process is re-parented
- Parent PID changes to the new parent (usually init/system process)

Race Condition

A race condition occurs when:

- Multiple processes access shared data
- Final result depends on execution order

Race Conditions with fork()

- After fork(), parent and child execute concurrently
- It is unpredictable which one runs first
- Logic depending on execution order can fail

Waiting and Polling

If a process wants to:

- Wait for a child → use wait()
- Wait for a parent → polling loop may be used

Problem with Polling

- Polling wastes CPU time
- Process wakes periodically just to check a condition

Solution

- Use signaling mechanisms instead of polling
- Signals avoid race conditions and CPU wastage

Life Cycle of an I/O Request – Overview

- An I/O request is made by a running process to access an input or output device.
- Input devices: keyboard, mouse, network
- Output devices: display, printer
- I/O operations are much slower than CPU operations.
- The operating system acts as an intermediary between processes and hardware.

The I/O life cycle defines the steps from request initiation to completion.

I/O Request Initiation and Kernel Processing

- Process initiates I/O using system calls:
 - read(), write(), open()
- Control switches from **user mode to kernel mode**
- Kernel:
 - Validates parameters
 - Checks file descriptors and permissions
- OS identifies:
 - Target device
 - Corresponding device driver
- Request is packaged into a kernel I/O data structure
- Request is placed in the device I/O queue
- Request may be:
 - Blocking
 - Non-blocking

Device Driver and I/O Execution

- Device driver:
 - Acts as a bridge between OS and hardware
 - Converts generic I/O request into device-specific commands
- Device controller starts the I/O operation
- Requesting process:
 - Is moved to Waiting (Blocked) state
- OS performs a context switch
- CPU executes other ready processes

This overlap improves CPU utilization and throughput

I/O Completion and Process Resumption

- Device completes the I/O operation
- Device generates an interrupt
- CPU transfers control to Interrupt Service Routine (ISR)
- ISR:
 - Verifies completion
 - Handles errors if any
- Blocked process moved to Ready state
- Scheduler selects the process
- Process resumes execution
- Control returns to user mode
- System call completes